

REPOSITORY ASSESSMENT

Video Downloader Project Audit

Security, reliability, dependency health, and operational review of the Next.js application and its download pipeline.

OVERALL RISK: HIGH FOR PUBLIC DEPLOYMENT

Bottom line. The application builds and passes lint, and it has useful host validation and resource controls. Do not expose a credential-enabled deployment publicly until dependencies are patched and caller access is restricted. The current endpoint shares configured account credentials across anonymous users, while the per-process concurrency ceiling is not a rate limit.

1 Critical

Production dependency advisory reported by npm audit

3 High

Credential isolation, cookie-file exposure, and public resource abuse

2 Passed

Configured lint check and optimized production build

0 Tests

No test script or automated behavior suite is defined

Review scope

Reviewed the Next.js API routes, UI components, browser history store, binary installers, package manifests, Docker and hosting configuration, ignore rules, and project documentation. Dependency findings reflect the registry advisory data returned by `npm audit --omit=dev` on the audit date.

No live downloads, authenticated platform sessions, deployed-host checks, or adversarial load tests were run. Secret values and cookie-file contents were not inspected.

Priority: patch Next.js/PostCSS, protect the public API with authentication and rate limits, and prevent exported cookies from entering Git.

Priority findings

F-01 Production framework dependencies have known advisories

CRITICAL

Evidence. `npm audit --omit=dev` reports a critical Next.js advisory and a high PostCSS advisory in the production dependency tree. The app pins Next.js 14.2.0; its lockfile includes Next's PostCSS 8.4.31, and the direct PostCSS dependency is 8.4.38.

Impact. The audit output associates the installed versions with multiple Next.js issues, including denial-of-service and authorization/cache-related advisories. Publicly reachable app routes increase exposure even where a particular vulnerable feature is not intentionally used.

Action. Upgrade Next.js and PostCSS to currently patched compatible releases, commit the regenerated lockfile, then rerun the full audit and build. Review the specific advisories and affected feature conditions before release.

Source: `package.json:14,23`; `package-lock.json:4548`; validation: `npm audit --omit=dev`.

F-02 Public callers share any configured account credentials

HIGH

There is no caller authentication on download submission or job polling. When platform cookie files or Meta Graph credentials are configured, every submitted URL is processed with the operator's account context. Anyone who can reach the service can consume those credentials' access and quota, and can request material visible to the configured account.

Action. Keep credential-enabled deployments private until authenticated and authorized. Prefer per-user credentials or disable cookie/Graph API fallbacks on a public service; use least-privilege tokens and monitor usage.

Source: `app/api/download/route.js:507-592` (unauthenticated submission); `:648-707` (shared cookies); Graph fallback in the same route.

F-03 Root-level exported cookies are not Git-ignored

HIGH

The server explicitly auto-discovers `cookies.txt` in the repository root. `.gitignore` excludes the `cookies/` directory, but not this root file; `git check-ignore cookies.txt` confirmed it is not ignored. Browser cookies are bearer-like session credentials.

Action. Add root cookie-export patterns to `.gitignore`, keep exports outside the repository, and scan Git history. If a real export was committed, revoke the session and remove the secret from history through the repository's approved process.

Source: `app/api/download/route.js:666-671`; `.gitignore:17`.

Reliability and abuse resistance

F-04 Public download API lacks caller rate limits

HIGH

The route permits three active jobs per process, but does not limit requests by IP or identity. An attacker can keep slots occupied or submit repeated jobs as slots free, driving bandwidth, CPU, temporary-disk use, and source-platform blocks. The 500 MB media limit is per job, not an overall quota.

Action. Add edge or application rate limits, queue and daily quotas, and request-body limits. Require authentication for deployments intended for a known audience; alert on repeated failures and resource saturation.

Source: `app/api/download/route.js`:210-222,507-592,699-707.

F-05 The documented 10-minute job cap is not an overall deadline

MEDIUM

The 10-minute timer kills an individual yt-dlp child. The retry helper can launch up to three child attempts, and YouTube refusal handling tries multiple player clients sequentially. The job-map TTL starts only after a job completes or fails; it does not bound total job runtime. A job can therefore occupy a slot substantially longer than the README's stated 10-minute cap.

Action. Establish one absolute deadline when the job is created, pass its remaining time through metadata, retries, and platform fallbacks, and cancel all child processes when it expires. Align the README with the enforced limit.

Source: `app/api/download/route.js`:33,78,413-444,757-765; documented cap: `README.md`:41-47.

F-06 Native binaries are fetched from floating release targets

MEDIUM

Install-time scripts fetch yt-dlp and FFmpeg from `releases/latest` URLs and execute/use the downloaded binaries without a pinned version or checksum/signature verification. This makes builds non-reproducible and trusts mutable third-party release assets. FFmpeg archives are also fully buffered in memory before extraction.

Action. Pin versions, verify publisher checksums/signatures, fail installation when required assets are invalid or absent, and stream large archives to disk with a size limit.

Source: `scripts/get-ytdlp.js`:22-25; `scripts/get-ffmpeg.js`:11-12,56-75.

F-07 Deployment health checks can pass with a broken download engine

MEDIUM

`/api/health` reports engine availability in JSON but returns HTTP 200 when yt-dlp is missing. Render's configured health check uses the HTTP status, so it can mark an unusable downloader healthy. Install scripts are best-effort and can leave the binary absent without failing installation.

Action. Separate liveness from readiness. Make readiness return a failing HTTP status when required engines are absent, or configure deployment monitoring to evaluate the JSON ok field.

Source: `app/api/health/route.js`:15-67; `render.yaml`:25-29; `scripts/get-ytdlp.js`:19-31.

Strengths, verification, and next steps

Good controls already present

- Validates URL protocols and restricts hostnames to supported platform domains before launching yt-dlp.
- Uses child-process argument arrays rather than shell interpolation and separates the URL after --.
- Caps active jobs per process, restricts playlists, caps requested resolution and file size, and cleans finished temporary files.
- Uses random job/download tokens and expires stored results; job state and local browser history are intentionally separated.
- Provides container deployment guidance and explicitly notes that in-memory jobs require a single persistent instance.

Verification performed

- `npm run lint`: passed with no warnings or errors.
- `npm run build`: passed; Next.js generated the production build and route manifest.
- `npm audit --omit=dev`: failed with 2 production dependency findings (1 critical, 1 high).
- No test script or unit/integration suite is defined in `package.json`.
- Working tree was clean before this report was created.

Recommended remediation order

- 1 Patch dependencies:** update Next.js and PostCSS to patched compatible versions, then rerun audit, lint, and build.
- 2 Restrict service access:** add authentication and rate limits before using cookies or Meta credentials on any public deployment.
- 3 Protect secrets:** ignore root cookie exports, keep secrets out of Git, and rotate any session that may have been exposed.
- 4 Enforce resource bounds:** add a total job deadline, request quotas/body limits, and a readiness check that fails when the engine is unavailable.
- 5 Harden builds:** pin and verify native binary artifacts; add focused tests for URL validation, job expiry, authorization, and cleanup.

Risk conclusion. Suitable for local experimentation after dependency updates. Public production use needs dependency remediation and access/credential controls first; current process limits and explanatory UI are not substitutes for tenant isolation or abuse prevention.

This is a static repository audit, not a penetration test or legal review. External platform behavior, binary publishers, registry advisories, and hosting limits can change after the audit date.